

Ros2 Day1 hw2 보고서

휴머노이드 수습단원 이주원

목차

1. 프로젝트 개요
2. 코드 구조
3. 파일 구조
4. 실행결과

1. 프로젝트 개요

이 프로젝트는 Ros2에 대한 전반적인 이해와 토픽 전송시 int, string, float, bool 메세지 형태에 관해 데이터를 주고 받는 것에 대한 과제이다. stg_msgs를 이용하여 C++ 패키지와 파이썬 패키지가 서로 토픽 통신 하는 과정이 포함 되어 있다.

또한 rqt graph를 이용하여 노드, 토픽 연결 구조 등을 보여준다는 점이 있다.

이 과제에서 토픽은 총 4개 이며 다음과 같은 토픽이 있다.

/my_bool_topic 부울형 관련 토픽
/my_float_topic 실수형 관련 토픽
/my_int_topic 정수형 관련 토픽
/my_str_topic 문자형 관련 토픽

또한 2개의 다음과 같은 노드가 서로 통신을 한다.

/my_publisher
/my_subscriber

이름에서도 알 수 있듯 퍼블리셔와 셉스크라이버가 서로 토픽을 주고 받는다.

2. 코드 구조

C++

헤더 파일

```
#include "std_msgs/msg/string.hpp"
#include "std_msgs/msg/int32.hpp"
#include "std_msgs/msg/float32.hpp"
#include "std_msgs/msg/bool.hpp"
```

.msg 파일로 정의된 메시지 타입이 C++ 구조체로 자동 변환된 헤더를 가져온다. 만약 없으면 std_msgs::msg::Int32 같은 타입을 코드를 쓸 수 있다.

```
void topic_callback(const std_msgs::msg::String::SharedPtr msg);
void int_callback(const std_msgs::msg::Int32::SharedPtr msg);
void float_callback(const std_msgs::msg::Float32::SharedPtr msg);
void bool_callback(const std_msgs::msg::Bool::SharedPtr msg);
```

메시지가 도착했을 때 실행될 콜백 함수들의 선언

```
rclcpp::Subscription<std_msgs::msg::String>::SharedPtr my_str_subscriber_;
rclcpp::Subscription<std_msgs::msg::Int32>::SharedPtr my_int_subscriber_;
rclcpp::Subscription<std_msgs::msg::Float32>::SharedPtr my_float_subscriber_;
rclcpp::Subscription<std_msgs::msg::Bool>::SharedPtr my_bool_subscriber_;
```

각 토픽을 구독하는 구독자 객체를 담은 멤버 변수

```
rclcpp::TimerBase::SharedPtr timer_;
void timer_callback();
```

주기적으로 콜백을 부를 타이머 객체와, 그 타이머가 부를 함수의 선언

```
rclcpp::Publisher<std_msgs::msg::Int32>::SharedPtr int_pub_;
rclcpp::Publisher<std_msgs::msg::String>::SharedPtr str_pub_;
rclcpp::Publisher<std_msgs::msg::Float32>::SharedPtr float_pub_;
rclcpp::Publisher<std_msgs::msg::Bool>::SharedPtr bool_pub_;
```

각 타입별 메시지를 발행할 퍼블리셔 객체를 담은 멤버 변수

cpp파일

```
count_ = 0;
str_pub_ = this->create_publisher<std_msgs::msg::String>("my_str_topic", 10);
int_pub_ = this->create_publisher<std_msgs::msg::Int32>("my_int_topic", 10);
float_pub_ = this->create_publisher<std_msgs::msg::Float32>("my_float_topic", 10);
bool_pub_ = this->create_publisher<std_msgs::msg::Bool>("my_bool_topic", 10);
timer_ = this->create_wall_timer(1s, std::bind(&MyPublisher::timer_callback, this));
```

헤더에 선언만 해둔 퍼블리셔 4개와 타이머를 실제로 생성해서 멤버 변수에 담는 부분

```
auto msg = std_msgs::msg::String();
auto int_msg = std_msgs::msg::Int32();
auto float_msg = std_msgs::msg::Float32();
auto bool_msg = std_msgs::msg::Bool();
```

각 타입의 빈 메시지 객체를 만드는 부분

```
msg.data = "Hello World: " + std::to_string(count_++);
int_msg.data = count_;
float_msg.data = static_cast<float>(count_) * 1.1f;
bool_msg.data = count_ % 2 == 0;
```

각 메시지의 data 필드에 값을 채우는 부분

```
str_pub_->publish(msg);
int_pub_->publish(int_msg);
float_pub_->publish(float_msg);
bool_pub_->publish(bool_msg);
```

채운 메시지를 각 퍼블리셔로 발행

```
RCLCPP_INFO(this->get_logger(), "Published message: '%s'", msg.data.c_str());
```

터미널에 로그

```
my_str_subscriber_ = this->create_subscription<std_msgs::msg::String>(
    "my_str_topic",
    10,
    std::bind(&MySubscriber::topic_callback, this, std::placeholders::_1));
```

my_str_topic 토픽을 구독하는 구독자를 만들어서 멤버 변수에 담는 부분

```
void MySubscriber::topic_callback(const std_msgs::msg::String::SharedPtr msg)
{
    RCLCPP_INFO(this->get_logger(), "Received message: '%s'", msg->data.c_str());
}
```

메시지가 도착했을 때 실제로 실행되는 콜백 함수의 구현

```
rclcpp::init(argc, argv);
auto node = std::make_shared<MySubscriber>();
rclcpp::spin(node);
rclcpp::shutdown();
return 0;
```

ROS2를 초기화하고, 노드 객체를 만들어서, spin으로 계속 돌리다가, 끝나면 정리하는 부분

Py

```
self.bool_pub = self.create_publisher(Bool, 'my_bool_topic', 10)
self.timer = self.create_timer(1, self.timer_callback)
self.count = 0
```

Bool 타입 퍼블리셔를 만들고, 1초마다 timer_callback을 부를 타이머를 만들고, 카운터를 0으로 초기화하는 부분

```
def timer_callback(self):
    str_msg = String()
```

1초마다 호출될 콜백 함수를 정의하고, 그 안에서 빈 String 메시지 객체를 만드는 부분

```
self.count += 1
str_msg.data = 'Hello from Python: {0}'.format(self.count)
int_msg.data = self.count
```

카운터를 1 올리고, 그 값으로 문자열과 정수 메시지의 data 필드를 채우는 부분

```
self.float_pub.publish(float_msg)
self.bool_pub.publish(bool_msg)
self.get_logger().info('Published: {0}'.format(str_msg.data))
```

값을 채운 메시지를 각 퍼블리셔로 발행하고, 마지막에 터미널에 로그를 찍는 부분

```

def main(args=None):
    rclpy.init(args=args)
    node = MyPublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        node.get_logger().info('Keyboard Interrupt (SIGINT)')
    finally:
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

ROS2를 초기화하고 노드를 만들어 spin으로 돌리다가, 끝나면 정리하는 부분

```

def __init__(self):
    super().__init__('my_py_subscriber')
    self.str_sub = self.create_subscription(
        String, 'my_str_topic', self.str_callback, 10)

```

노드를 my_py_subscriber라는 이름으로 초기화하고, my_str_topic을 구독하는 구독자를 만들어 self.str_sub에 담는 부분

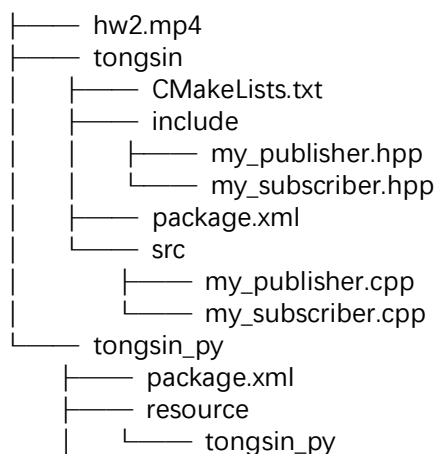
```

def str_callback(self, msg):
    self.get_logger().info('Received string: {0}'.format(msg.data))

```

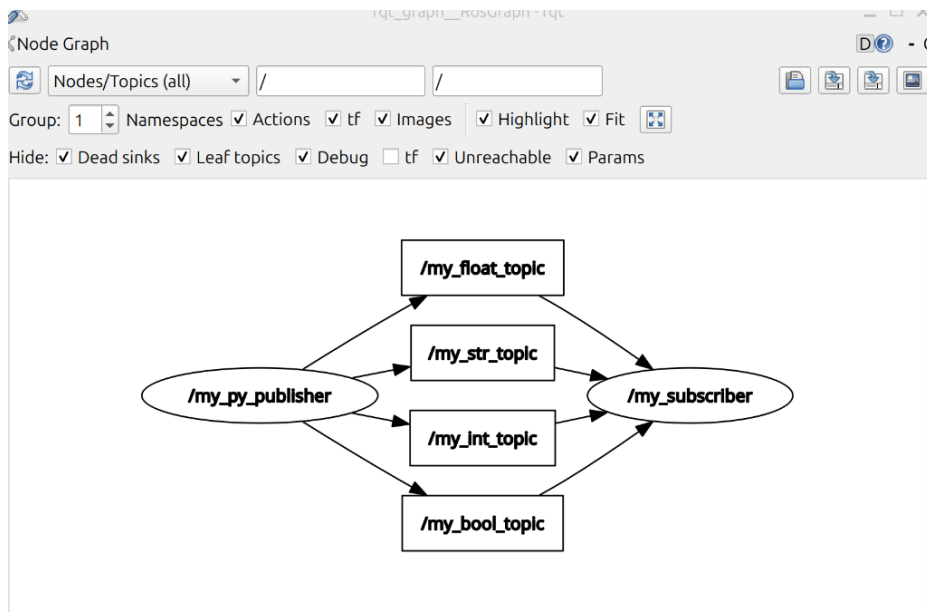
메시지가 도착했을 때 실행되는 콜백 함수로, 받은 메시지의 data 값을 터미널에 찍음

3. 파일 구조



- setup.cfg
- setup.py
- test
 - test_copyright.py
 - test_flake8.py
 - test_pep257.py
- tongsin_py
 - _init_.py
 - my_publisher.py
 - my_subscriber.py

4. 실행 결과



실행 했을 때의 노드와 토픽을 rqt_graph로 나타낸 것이다. 파이썬의 퍼블리셔에서 앞서 설명한 네 개의 토픽이 cpp의 섭스크라이브로 가는 구조이다.

```

[INFO] [1788884563.078758093] [my_subscriber]: Received message: 't
ue'
[INFO] [1788884563.078858431] [my_subscriber]: Received message: '1
54.400024'
[INFO] [1788884563.078898511] [my_subscriber]: Received message: '1
504'
[INFO] [1788884563.078919455] [my_subscriber]: Received message: 'H
ello from Python: 1504'
[INFO] [1788884564.079925373] [my_subscriber]: Received message: '1
55.500000'
[INFO] [1788884564.080088514] [my_subscriber]: Received message: '1
505'
[INFO] [1788884564.080121040] [my_subscriber]: Received message: 'H
ello from Python: 1505'
[INFO] [1788884564.080187845] [my_subscriber]: Received message: 'f
alse'

py, (line 129, in shutdown
self.__context.shutdown()
rcipy._rcipy_pybind11.RCLError: failed to shutdown: rcl_shutdown a
ready called on the given context, at ./src/rcl/init.c:333
[rosrun]: Process exited with failure 1
ray@ray:~/Documents/21th_robot_intern_LeeJuwon/ROS2/day15 ros2 top
ic list
/my_bool_topic
/my_float_topic
/my_int_topic
/my_str_topic
/parameter_events
/rosout
ray@ray:~/Documents/21th_robot_intern_LeeJuwon/ROS2/day15 rqt_grap
SocketNotifier: Can only be used with threads started with QThrea
ayland does not support QWindow::requestActivate()
  
```